

Backend Architecture Specification

Hono · Drizzle · Neon · Better Auth · Zod · Cloudflare Workers

Document version 1.0 **Target runtime** Cloudflare Workers (V8 isolates) **Status** Reference architecture

Table of Contents

Part I — Foundations 1. Architectural Philosophy 2. Dependency Injection: The Decision 3. Directory Structure 4. Environment & Configuration

Part II — Wiring 5. The Composition Root 6. Database Layer 7. Repository Pattern 8. Service Layer

Part III — HTTP Concerns 9. Middleware Pipeline 10. Authentication & Bearer Tokens 11. Authorization 12. Error Handling 13. Pagination 14. OpenAPI & Route Definitions

Part IV — Operations 15. Testing Strategy 16. Deployment 17. Observability 18. Migration Discipline

Appendices - A. Full Package Manifest - B. Framework Comparison Matrix - C. Decision Log

Part I — Foundations

1. Architectural Philosophy

1.1 Core Principles

This architecture is governed by five non-negotiable principles. Every subsequent decision in this document derives from one of them.

****P1 — Explicit over implicit.**** If a module depends on something, that dependency arrives as a function argument. No module-level singletons. No ambient imports of live connections. No decorators that resolve things at runtime through metadata reflection. You should be able to read a file top-to-bottom and know exactly what it touches.

****P2 — The transport layer is thin.**** An HTTP handler parses input, calls one service method, and serializes output. It contains no ``if`` statements about business rules. This is Go's ``http.Handler`` philosophy and it is correct. If your handler is more than ten lines, logic has leaked upward.

****P3 — Vertical slices, not horizontal layers.**** A feature lives in one directory. Adding an endpoint should touch one folder, not five. Laravel's ``app/Http/Controllers`` + ``app/Models`` + ``app/Services`` split forces you to navigate the codebase by *technical role* rather than by *business concern*. Django apps and Go packages get this right.

****P4 — Schema is the single source of truth.**** The database schema generates the Zod validators, which generate the OpenAPI spec, which generates the TypeScript types. One definition, four consumers. Drift becomes structurally impossible rather than merely discouraged.

****P5 — Business logic must run without HTTP.**** A service method must be callable from a cron trigger, a queue consumer, a CLI script, or a unit test. If a service accepts a ``Context`` object, HTTP has contaminated your domain.

1.2 What We Take From Established Frameworks

The point of studying mature frameworks is to steal their hard-won lessons while leaving behind the accidental complexity they accumulated solving problems specific to their era and runtime.

Framework	What to take	What to leave
NestJS	Module boundaries; strict layer separation; the discipline of a defined dependency graph	DI container; decorators; <code>reflect-metadata</code> ; provider scopes
Django	Apps as vertical slices; fat models / thin views; migrations as first-class artifacts	ORM calls inside views; signals; implicit <code>settings</code> global
Laravel	Form Requests ($\approx$ our Zod schemas); migration discipline; explicit route files	Facades; global service container; <code>helper()</code> functions everywhere
Express	Middleware as composable functions; minimal core	The total absence of structure; callback-era error handling
Go (stdlib + chi)	Explicit dependency passing; interfaces defined by the consumer; no magic; errors as values	Verbosity as a virtue; lack of a validation ecosystem
Spring Boot	Configuration validation at boot; clear bean lifecycle concepts	XML/annotation-driven wiring; classpath scanning

1.3 Why Cloudflare Workers Changes the Calculus

Several patterns that are load-bearing on a long-lived Node process are actively harmful in a V8 isolate:

- Isolates are ephemeral.** There is no meaningful application-lifetime singleton. A DI container that resolves and caches singletons is caching into a scope that may vanish between requests.
- Cold starts are measured in single-digit milliseconds.** Anything that scans, reflects, or builds a metadata graph at startup is pure tax paid on every cold isolate.
- There is no `reflect-metadata` without polyfill cost.** Decorator-based DI requires emitting and reading design-time type metadata. This inflates bundle size and adds startup work.
- The dependency graph is shallow.** In practice: `env  $\rightarrow$  db  $\rightarrow$  repository  $\rightarrow$  service  $\rightarrow$  handler`. Four levels. A container solves a problem that appears at depth 15, not depth 4.

The result: we keep the idea of dependency injection — dependencies flow inward as arguments — and discard the machinery of a DI container.

2. Dependency Injection: The Decision

2.1 Verdict

****Do not use a DI container. Do use dependency injection.**** These are not the same thing, and conflating them is the single most common architectural mistake in the TypeScript ecosystem.

Dependency injection is a *principle*: a component does not construct or locate its own dependencies; it receives them.

A DI container is one *implementation* of that principle: a runtime registry that resolves the dependency graph on your behalf, usually via reflection, decorators, or string tokens.

You want the principle. You do not need the implementation.

2.2 The Failure Mode We Are Avoiding

```
// ANTI-PATTERN: hidden global dependency
import { db } from '@db/client'

export async function getUser(id: string) {
  return db.select().from(users).where(eq(users.id, id))
}
```

This function is:

- **Untestable in isolation.** To test it you must mock the module system itself (`vi.mock('@db/client')`), which is a test of your bundler, not your logic.
- **Opaque.** Its signature says `(id: string) => Promise<User>` . It lies. It actually requires a live database connection.
- **Coupled to instantiation order.** The module-level `db` must be initialized before this module is imported. On Workers, `env` is not available at module scope — this code cannot even *run*.

2.3 The Pattern We Use: Factory Functions with Closed-Over Dependencies

```
// user.service.ts

export interface UserServiceDeps {
  userRepo: UserRepository
  logger: Logger
}

export function makeUserService(deps: UserServiceDeps) {
  const { userRepo, logger } = deps

  return {
    async getById(id: string): Promise<User> {
      const user = await userRepo.findById(id)
      if (!user) throw new NotFoundError('User')
      return user
    },

    async list(params: PaginationParams): Promise<Paginated<User>> {
      logger.debug('listing users', { params })
      return userRepo.paginate(params)
    }
  }
}

export type UserService = ReturnType<typeof makeUserService>
```

Note what this buys us:

1. **The signature is honest.** `makeUserService` announces it needs a `userRepo` and a `logger` .
2. **The return type is derived.** `ReturnType<typeof makeUserService>` means the interface can never drift from the implementation. No separate `IService` to keep in sync.
3. **Testing requires no framework.**

```
const svc = makeUserService({
  userRepo: { findById: async () => fixtures.user } as UserRepository,
  logger: noopLogger,
})

expect(await svc.getById('x')).toEqual(fixtures.user)
```

No `Test.createTestingModule()` . No `@Injectable()` . No test container. If a service is hard to test, it is because a dependency was not injected — the pain is diagnostic.

2.4 Classes vs. Factory Functions

Both satisfy P1. The choice is stylistic, with one real consideration:

```
// Class form — equally valid
export class UserService {
  constructor(private readonly deps: UserServiceDeps) {}

  async getById(id: string): Promise<User> { /* ... */ }
}
```

Prefer factory functions when: you want structural typing, tree-shakeable exports, and zero `this` binding hazards when methods are passed as callbacks.

Prefer classes when: your team has strong OO muscle memory, or you want `private` field enforcement at the type level.

Do not mix the two arbitrarily. Pick one and put it in the linter.

2.5 When You *Would* Want a Container

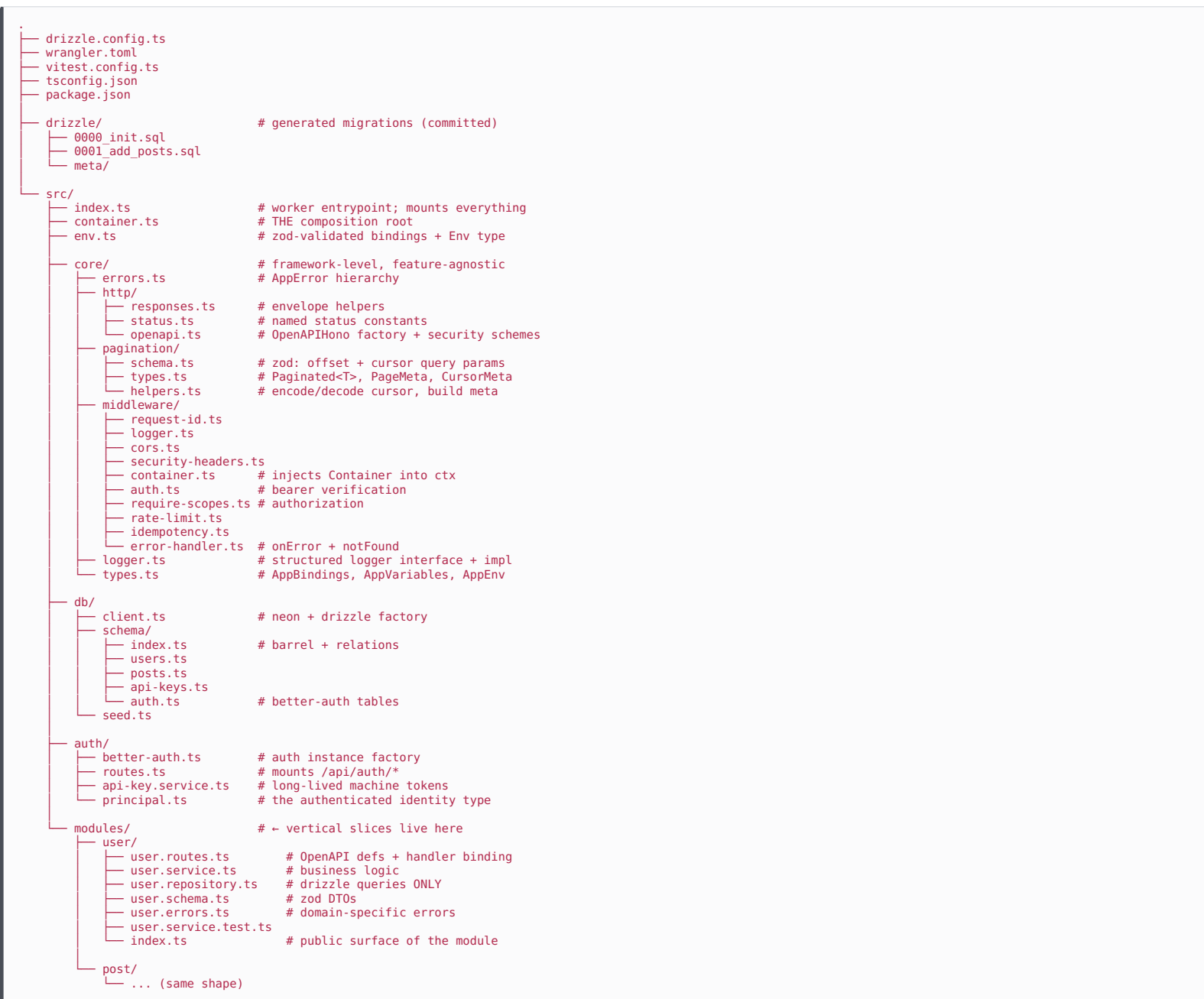
For completeness — a container earns its keep when:

- The dependency graph exceeds ~30 nodes with non-trivial ordering constraints.
- You need runtime-swappable implementations chosen by configuration (multi-tenant with per-tenant storage backends).
- You need lifecycle hooks (`onModuleInit` , `onApplicationShutdown`) across many components.
- You genuinely need lazy resolution of circular dependencies.

If you hit three of these, revisit. Until then, `container.ts` — a single file, twenty lines, plain function calls — is strictly better because you can *read* it.

3. Directory Structure

3.1 The Tree



3.2 Rules of the Structure

****R1** — A module is a deletable unit.** Deleting ``modules/post/`` and its route registration should leave a compiling, working application. If it doesn't, you have a leaked dependency.

****R2** — Modules communicate through services, never through repositories.** ``PostService`` may depend on ``UserService``. ``PostService`` may **never** depend on ``UserRepository``. The service is the module's contract; the repository is its private implementation.

****R3** — ``core/`` never imports from ``modules/``.** Dependency direction is strictly inward. ``core`` knows nothing about your business. Enforce with an ESLint boundary rule.

****R4** — Each module exports through ``index.ts``.****** The barrel defines the public surface. Anything not exported there is internal. This is Go's capitalization convention, expressed in TypeScript.

3.3 Layer Contracts

This table is the most important thing in this document. Violations of it are how architectures rot.

Layer	May import	Must never touch	Returns
Routes	Zod schemas, service types, <code>core/http</code>	Drizzle, <code>db/</code> , repositories, SQL	<code>Response</code>
Service	Repository interfaces, other services, domain errors	<code>Context</code> , <code>c.req</code> , <code>Response</code> , HTTP status codes	Domain objects; throws domain errors
Repository	Drizzle, schema, SQL	Business rules, HTTP, other repositories	Rows / <code>null</code>
Schema (db)	Drizzle column builders	Everything else	—

The `Context` test. If you can grep your `src/modules/**/*.service.ts` for `Context` , `c.req` , `c.json` , or `HTTPException` and get zero results, your service layer is clean. Put that grep in CI.

4. Environment & Configuration

4.1 Fail Fast, Fail Loudly

Spring Boot validates configuration at startup and refuses to boot on a bad value. We do the same, at first request, with Zod. A misconfigured worker should return `500` immediately with a clear log line — never limp along and fail mysteriously three layers deep.

4.2 `src/env.ts`

```
import { z } from 'zod'

/**
 * Cloudflare bindings, as declared in wrangler.toml.
 * These arrive untyped and unvalidated on `c.env`.
 */
export const EnvSchema = z.object({
  // --- secrets (wrangler secret put) ---
  DATABASE_URL: z.string().url().startsWith('postgres'),
  BETTER_AUTH_SECRET: z.string().min(32),
  API_KEY_PEPPER: z.string().min(32),

  // --- vars (wrangler.toml [vars]) ---
  ENVIRONMENT: z.enum(['development', 'preview', 'production']),
  BETTER_AUTH_URL: z.string().url(),
  LOG_LEVEL: z.enum(['debug', 'info', 'warn', 'error']).default('info'),
  CORS_ORIGINS: z
    .string()
    .transform((s) => s.split(',').map(o => o.trim()))
    .pipe(z.array(z.string().url())),

  // --- pagination policy ---
  PAGE_SIZE_DEFAULT: z.coerce.number().int().min(1).max(200).default(20),
  PAGE_SIZE_MAX: z.coerce.number().int().min(1).max(500).default(100),
})

export type Env = z.infer<typeof EnvSchema>

/**
 * Raw bindings type — what Cloudflare actually hands us.
 * Includes non-serializable bindings that Zod cannot describe.
 */
export type RawBindings = Record<string, unknown> & {
  RATE_LIMIT_KV: KVNamespace
  IDEMPOTENCY_KV: KVNamespace
}

export function parseEnv(raw: unknown): Env {
  const result = EnvSchema.safeParse(raw)
  if (!result.success) {
    // Flatten produces a readable, greppable log line.
    const issues = result.error.issues
      .map((i) => `${i.path.join('.')}: ${i.message}`)
      .join('; ')
    throw new Error(`[env] Invalid configuration — ${issues}`)
  }
  return result.data
}
```

4.3 Why Not `process.env` ?

Workers has no `process`. Bindings arrive per-request on `c.env`. This is a feature: it means configuration is *injected*, which is exactly principle P1. Do not fight it by hoisting `env` into a module-level variable — that reintroduces the global you just eliminated.

4.4 `wrangler.toml`

```
name = "api"
main = "src/index.ts"
compatibility_date = "2025-01-01"
compatibility_flags = ["nodejs_compat"]

[vars]
ENVIRONMENT = "development"
BETTER_AUTH_URL = "http://localhost:8787"
LOG_LEVEL = "debug"
CORS_ORIGINS = "http://localhost:3000"
PAGE_SIZE_DEFAULT = "20"
PAGE_SIZE_MAX = "100"

[[kv_namespaces]]
binding = "RATE_LIMIT_KV"
id = "..."

[[kv_namespaces]]
binding = "IDEMPOTENCY_KV"
id = "..."

[env.production]
[env.production.vars]
ENVIRONMENT = "production"
BETTER_AUTH_URL = "https://api.example.com"
LOG_LEVEL = "info"
CORS_ORIGINS = "https://example.com,https://app.example.com"

# Secrets are NEVER in this file.
# wrangler secret put DATABASE_URL
# wrangler secret put BETTER_AUTH_SECRET
# wrangler secret put API_KEY_PEPPER
```

4.5 Application Types

```
// core/types.ts
import type { Env, RawBindings } from '@env'
import type { Container } from '@container'
import type { Principal } from '@auth/principal'

export type AppVariables = {
  container: Container
  requestId: string
  principal: Principal | null
  logger: Logger
}

export type AppEnv = {
  Bindings: RawBindings
  Variables: AppVariables
}
```

Every `Hono` and `OpenAPIHono` instance in the codebase is parameterized with `AppEnv`. This gives you fully typed `c.get('container')` with zero casts.

Part II — Wiring

5. The Composition Root

5.1 The Concept

Mark Seemann's term. **The composition root is the single place in an application where the object graph is assembled.** It should be one file, near the entrypoint, and it should be the *only* place that calls constructors or factories for stateful components.

Everything else in the codebase receives its dependencies. Only the composition root creates them.

5.2 `src/container.ts`

```
import { parseEnv, type RawBindings } from '@env'
import { createDb } from '@db/client'
import { createLogger } from '@core/logger'
import { createAuth } from '@auth/better-auth'
import { makeApiKeyService } from '@auth/api-key.service'

import { makeUserRepository } from '@modules/user/user.repository'
import { makeUserService } from '@modules/user/user.service'
import { makePostRepository } from '@modules/post/post.repository'
import { makePostService } from '@modules/post/post.service'

export interface ContainerContext {
  requestId: string
}

export function createContainer(raw: RawBindings, ctx: ContainerContext) {
  // 1. Validate configuration. Throws on bad config.
  const env = parseEnv(raw)

  // 2. Infrastructure
  const logger = createLogger({
    level: env.LOG_LEVEL,
    requestId: ctx.requestId,
    environment: env.ENVIRONMENT,
  })
  const db = createDb(env.DATABASE_URL)
  const auth = createAuth(env, db)

  // 3. Repositories – depend only on `db`
  const userRepo = makeUserRepository(db)
  const postRepo = makePostRepository(db)
  const apiKeyRepo = makeApiKeyRepository(db)

  // 4. Services – depend on repositories and each other
  const apiKeyService = makeApiKeyService({
    apiKeyRepo,
    pepper: env.API_KEY_PEPPEER,
  })

  const userService = makeUserService({ userRepo, logger })

  const postService = makePostService({
    postRepo,
    userService, // ← service→service, never service→foreign repo
    logger,
  })

  return {
    env,
    logger,
    db,
    auth,
    kv: {
      rateLimit: raw.RATE_LIMIT_KV,
      idempotency: raw.IDEMPOTENCY_KV,
    },
    services: {
      user: userService,
      post: postService,
      apiKey: apiKeyService,
    },
  }
}

export type Container = ReturnType<typeof createContainer>
```

5.3 Read That Again

Twenty-five lines. You can see:

- what configuration exists,
- what infrastructure is created,
- exactly which repository each service receives,
- exactly which services depend on which other services.

In NestJS this same information is distributed across `app.module.ts`, `user.module.ts`, `post.module.ts`, three `@Injectable()` decorators, and a set of `imports`: `exports`: arrays. You cannot see the graph. You can only trust that the framework built it.

****This is the payoff.**** Not performance, not bundle size — **legibility**. A new engineer reads one file and understands the system's shape.

5.4 Per-Request Instantiation and Its Caveat

The container is constructed **per request**:

```
// core/middleware/container.ts
import { createMiddleware } from 'hono/factory'
import { createContainer } from '@/container'
import type { AppEnv } from '@core/types'

export const containerMiddleware = createMiddleware<AppEnv>({
  async (c, next) => {
    const container = createContainer(c.env, {
      requestId: c.get('requestId'),
    })
    c.set('container', container)
    c.set('logger', container.logger)
    await next()
  },
})
```

This is safe **because Neon's HTTP driver is stateless**. `createDb()` builds a query builder, not a connection. There is no socket, no pool, no handshake. The cost is a few object allocations.

****Caveat — connection state.**** If you switch to Neon's WebSocket driver (`@neondatabase/serverless`Pool``), or add any dependency with connection state or an expensive constructor, **do not build it per request**. Hoist that single component to module scope, guard it with a lazy singleton, and pass it into `createContainer`. Keep the **services** per-request. Cache the **connections**.

```
// db/pool.ts — ONLY if you need WebSocket/pooled connections
let _pool: Pool | undefined

export function getPool(url: string): Pool {
  if (!_pool) _pool = new Pool({ connectionString: url })
  return _pool
}

// container.ts
export function createContainer(raw: RawBindings, ctx: ContainerContext) {
  const env = parseEnv(raw)
  const db = drizzle(getPool(env.DATABASE_URL)) // pooled, cached
  // ...services still built fresh
}
```

The rule: **cache what is expensive and stateless-to-share; rebuild what is cheap and request-scoped**.

6. Database Layer

6.1 Client Factory

```
// db/client.ts
import { drizzle } from 'drizzle-orm/neon-http'
import { neon } from '@neondatabase/serverless'
import * as schema from './schema'

export function createDb(connectionString: string) {
  const sql = neon(connectionString)
  return drizzle(sql, {
    schema,
    logger: false, // enable per-env via container if desired
  })
}

export type Database = ReturnType<typeof createDb>
```

`Database` is the type every repository takes. It is exported once and imported everywhere — but as a **type**, not a value. Types have no runtime existence and therefore cannot be a hidden global.

6.2 Schema Definition

```
// db/schema/users.ts
import {
  pgTable, uuid, text, timestamp, boolean, index, uniqueIndex,
} from 'drizzle-orm/pg-core'

export const users = pgTable(
  'users',
  {
    id: uuid('id').primaryKey().defaultRandom(),
    email: text('email').notNull(),
    name: text('name').notNull(),
    emailVerified: boolean('email_verified').notNull().default(false),
    avatarUrl: text('avatar_url'),

    createdAt: timestamp('created_at', { withTimezone: true })
      .notNull()
      .defaultNow(),
    updatedAt: timestamp('updated_at', { withTimezone: true })
      .notNull()
      .defaultNow()
      .$onUpdate(() => new Date()),
    deletedAt: timestamp('deleted_at', { withTimezone: true }),
  },
  (t) => ({
    emailIdx: uniqueIndex('users_email_idx').on(t.email),

    // CRITICAL for keyset pagination — see §13.4
    cursorIdx: index('users_cursor_idx').on(t.createdAt, t.id),
  })),
)

export type UserRow = typeof users.$inferSelect
export type NewUserRow = typeof users.$inferInsert
```

****The `(createdAt, id)` composite index is not optional.**** Cursor pagination performs `WHERE (created\_at, id) < (\$1, \$2) ORDER BY created\_at DESC, id DESC`. Without a matching composite index, Postgres will sort the entire table on every page request. This is the difference between a 2ms query and a 2-second query at 10M rows.

6.3 Relations

```
// db/schema/index.ts
import { relations } from 'drizzle-orm'
import { users } from './users'
import { posts } from './posts'

export * from './users'
export * from './posts'
export * from './api-keys'
export * from './auth'

export const usersRelations = relations(users, ({ many }) => ({
  posts: many(posts),
}))

export const postsRelations = relations(posts, ({ one }) => ({
  author: one(users, {
    fields: [posts.authorId],
    references: [users.id],
  }),
}))
```

6.4 Soft Deletes

`deletedAt` is nullable. Repositories that must respect it do so **explicitly**, in a shared predicate:

```
// db/predicates.ts
import { isNull } from 'drizzle-orm'
import { users } from './schema'

export const notDeleted = <T extends { deletedAt: PgColumn }>(t: T) =>
  isNull(t.deletedAt)
```

Do not use a global Drizzle filter or a query-builder wrapper that silently appends `WHERE deleted_at IS NULL`. That is implicit behavior, and the day you need to query deleted rows for an audit, you will fight the abstraction. Write the predicate. It is four characters.

7. Repository Pattern

7.1 Responsibilities

A repository:

- **Does** contain Drizzle queries and SQL.
- **Does** translate rows to domain shapes (or return rows directly if they match).
- **Does not** contain business rules, validation, authorization, or HTTP.
- **Does not** throw domain errors. It returns `null` or an empty array. Interpretation is the service's job.

That last point matters. `findById` returning `null` is a *fact*. Deciding that `null` means `404` is a *policy*, and policy belongs one layer up.

7.2 Implementation

```
// modules/user/user.repository.ts
import { eq, and, desc, lt, or, sql, isNull, count } from 'drizzle-orm'
import type { Database } from '@db/client'
import { users, type UserRow, type NewUserRow } from '@db/schema'
import type { CursorParams, OffsetParams } from '@core/pagination/types'

export function makeUserRepository(db: Database) {
  return {
    async findById(id: string): Promise<UserRow | null> {
      const [row] = await db
        .select()
        .from(users)
        .where(and(eq(users.id, id), isNull(users.deletedAt)))
        .limit(1)
      return row ?? null
    },

    async findByEmail(email: string): Promise<UserRow | null> {
      const [row] = await db
        .select()
        .from(users)
        .where(and(eq(users.email, email), isNull(users.deletedAt)))
        .limit(1)
      return row ?? null
    },

    async create(data: NewUserRow): Promise<UserRow> {
      const [row] = await db.insert(users).values(data).returning()
      return row
    },

    async update(id: string, patch: Partial<NewUserRow>): Promise<UserRow | null> {
      const [row] = await db
        .update(users)
        .set(patch)
        .where(and(eq(users.id, id), isNull(users.deletedAt)))
        .returning()
      return row ?? null
    },

    async softDelete(id: string): Promise<boolean> {
      const result = await db
        .update(users)
        .set({ deletedAt: new Date() })
        .where(and(eq(users.id, id), isNull(users.deletedAt)))
        .returning({ id: users.id })
      return result.length > 0
    },
  }

  // --- Pagination: see §13 for full treatment ---

  async paginateOffset(p: OffsetParams): Promise<{ rows: UserRow[]; total: number }> {
    const where = isNull(users.deletedAt)

    const [rows, [{ value: total }]] = await Promise.all([
      db.select().from(users)
        .where(where)
        .orderBy(desc(users.createdAt), desc(users.id))
        .limit(p.limit)
        .offset((p.page - 1) * p.limit),
      db.select({ value: count() }).from(users).where(where),
    ])

    return { rows, total }
  },

  async paginateCursor(p: CursorParams): Promise<UserRow[]> {
    const conditions = [isNull(users.deletedAt)]

    if (p.cursor) {
      // Keyset predicate. Row-value comparison — index-friendly.
      conditions.push(
        or(
          lt(users.createdAt, p.cursor.createdAt),
          and(
            eq(users.createdAt, p.cursor.createdAt),
            lt(users.id, p.cursor.id),
          ),
        ),
      )
    }

    return db
      .select()
      .from(users)
      .where(and(...conditions))
      .orderBy(desc(users.createdAt), desc(users.id))
      .limit(p.limit + 1) // +1 to detect hasNextPage
  },
}

export type UserRepository = ReturnType<typeof makeUserRepository>
```

7.3 On Extracting Interfaces

You have two options for the repository's type:

Option A — derive it (recommended to start):

```
export type UserRepository = ReturnType<typeof makeUserRepository>
```

Option B — declare it:

```
export interface UserRepository {  
  findById(id: string): Promise<UserRow | null>  
  // ...  
}  
export function makeUserRepository(db: Database): UserRepository { /* ... */ }
```

****Start with Option A. Migrate to Option B the first time you actually need a second implementation.****

Option B buys swappability — an in-memory repo for tests, a read-replica repo, a caching decorator. Option A buys speed and guarantees the type never drifts from the implementation.

Premature interface extraction is a real cost: you now maintain two definitions and must remember to update both. Note that Option A does *not* prevent testing — a structural type is satisfied by any object with matching members, so `{ findById: async () => fixture } as UserRepository` type-checks fine.

The Go instinct is correct here: **interfaces are defined by the consumer, at the moment a second implementation appears.**

8. Service Layer

8.1 The Contract

The service layer is where your application actually lives. It:

- **Owns business rules and invariants.**
- **Owns transaction boundaries.**
- **Throws domain errors** (`NotFoundError` , `ConflictError`) — never `HTTPException` .
- **Accepts and returns plain objects** — never `Context` , never `Response` .
- **Is callable from anywhere** — HTTP handler, cron trigger, queue consumer, test.

8.2 Implementation

```

// modules/user/user.service.ts
import type { UserRepository } from '../user.repository'
import type { Logger } from '@core/logger'
import { NotFoundError, ConflictError } from '@core/errors'
import { toCursorMeta, decodeCursor } from '@core/pagination/helpers'
import type { OffsetQuery, CursorQuery } from '@core/pagination/schema'
import type { Paginated, CursorPaginated } from '@core/pagination/types'
import type { CreateUserInput, UpdateUserInput, User } from '../user.schema'
import { toPublicUser } from '../user.schema'

export interface UserServiceDeps {
  userRepo: UserRepository
  logger: Logger
}

export function makeUserService({ userRepo, logger }: UserServiceDeps) {
  return {
    async getById(id: string): Promise<User> {
      const row = await userRepo.findById(id)
      if (!row) throw new NotFoundError('User', id)
      return toPublicUser(row)
    },

    async create(input: CreateUserInput): Promise<User> {
      // Business rule: email uniqueness.
      // Enforced here AND by a DB constraint. Belt and braces.
      const existing = await userRepo.findByEmail(input.email)
      if (existing) {
        throw new ConflictError('A user with this email already exists', {
          field: 'email',
        })
      }

      const row = await userRepo.create({
        email: input.email.toLowerCase().trim(),
        name: input.name.trim(),
      })

      logger.info('user.created', { userId: row.id })
      return toPublicUser(row)
    },

    async update(id: string, patch: UpdateUserInput): Promise<User> {
      const row = await userRepo.update(id, patch)
      if (!row) throw new NotFoundError('User', id)

      logger.info('user.updated', { userId: id, fields: Object.keys(patch) })
      return toPublicUser(row)
    },

    async remove(id: string): Promise<void> {
      const ok = await userRepo.softDelete(id)
      if (!ok) throw new NotFoundError('User', id)
      logger.info('user.deleted', { userId: id })
    },

    // --- Offset pagination ---
    async listOffset(q: OffsetQuery): Promise<Paginated<User>> {
      const { rows, total } = await userRepo.paginateOffset({
        page: q.page,
        limit: q.limit,
      })

      return {
        data: rows.map(toPublicUser),
        meta: {
          page: q.page,
          limit: q.limit,
          total,
          totalPages: Math.ceil(total / q.limit),
          hasNextPage: q.page * q.limit < total,
          hasPrevPage: q.page > 1,
        },
      }
    },

    // --- Cursor pagination ---
    async listCursor(q: CursorQuery): Promise<CursorPaginated<User>> {
      const rows = await userRepo.paginateCursor({
        limit: q.limit,
        cursor: q.cursor ? decodeCursor(q.cursor) : null,
      })

      // Repo fetched limit+1. Trim and report.
      const hasMore = rows.length > q.limit
      const page = hasMore ? rows.slice(0, q.limit) : rows

      return {
        data: page.map(toPublicUser),
        meta: toCursorMeta(page, hasMore),
      }
    },
  }
}

export type UserService = ReturnType<typeof makeUserService>

```

8.3 Transactions

Transaction boundaries belong to the service, because only the service knows what constitutes a single business operation. The repository must not open transactions — it does not have the context to know where one should end.

```
// The service receives `db` alongside its repos when it needs transactions.
export function makePostService({ db, postRepo, userRepo, logger }: PostServiceDeps) {
  return {
    async publishAndNotify(postId: string, authorId: string) {
      return db.transaction(async (tx) => {
        // Repos accept an optional executor, defaulting to `db`.
        const post = await postRepo.markPublished(postId, tx)
        if (!post) throw new NotFoundError('Post', postId)

        await userRepo.incrementPublishedCount(authorId, tx)

        // Anything thrown here rolls back both writes.
        return post
      })
    },
  }
}
```

To make this work, repository methods take an optional executor:

```
// modules/post/post.repository.ts
type Executor = Database | Parameters<Parameters<Database['transaction']>[0]>[0]

export function makePostRepository(db: Database) {
  return {
    async markPublished(id: string, tx: Executor = db) {
      const [row] = await tx
        .update(posts)
        .set({ publishedAt: new Date() })
        .where(eq(posts.id, id))
        .returning()
      return row ?? null
    },
  }
}
```

Neon HTTP driver does not support interactive transactions. `neon-http` supports only *batched* transactions (`db.batch([...])`) — an array of statements sent together. If you need `db.transaction(async tx => ...)` with conditional logic between statements, you must use the **WebSocket driver** (`drizzle-orm/neon-serverless` + `Pool`), which changes your connection caching strategy per §5.4. Decide this up front. It is the single most consequential infra choice in this stack.

Need	Driver	Transactions	Connection handling
Simple CRUD, independent writes	neon-http	Batched only	Stateless; build per request
Multi-step writes with conditional logic	neon-serverless (WebSocket)	Full interactive	Pooled; cache at module scope

Part III — HTTP Concerns

9. Middleware Pipeline

9.1 Ordering Is a Contract

Middleware order is not a style choice. It is a correctness requirement. Each layer depends on invariants established by the layers above it.

1. requestId	establishes trace identity
2. logger	needs requestId
3. errorHandler	must wrap everything below
4. securityHeaders	cheap, unconditional
5. cors	must run before auth (preflight!)
6. container	everything below needs services
7. rateLimit	needs KV from container
———— public routes terminate here ————	
8. auth	needs container (db, auth instance)
9. requireScopes	needs principal from auth
10. idempotency	needs principal for key scoping
———— protected route handlers ————	

****CORS must precede auth.**** A browser preflight (`OPTIONS`) carries no `Authorization` header. If auth runs first, every preflight returns `401`, and the browser never sends the real request. The failure is silent and maddening to debug.

****Rate limiting must precede auth.**** Otherwise an attacker exhausts your database with token-verification queries before the rate limiter ever sees them. Rate limit on IP for unauthenticated traffic; re-key on principal ID once identified.

9.2 Request ID

```
// core/middleware/request-id.ts
import { createMiddleware } from 'hono/factory'
import type { AppEnv } from '@core/types'

const HEADER = 'x-request-id'

export const requestId = createMiddleware<AppEnv>(async (c, next) => {
  // Honor upstream trace IDs (load balancer, gateway, CF ray).
  const incoming = c.req.header(HEADER) ?? c.req.header('cf-ray')
  const id = incoming ?? crypto.randomUUID()

  c.set('requestId', id)
  c.header(HEADER, id)
  await next()
})
```

9.3 Structured Logger

```
// core/logger.ts
export type LogLevel = 'debug' | 'info' | 'warn' | 'error'

const RANK: Record<LogLevel, number> = { debug: 10, info: 20, warn: 30, error: 40 }

export interface Logger {
  debug(msg: string, meta?: Record<string, unknown>): void
  info(msg: string, meta?: Record<string, unknown>): void
  warn(msg: string, meta?: Record<string, unknown>): void
  error(msg: string, meta?: Record<string, unknown>): void
  child(bindings: Record<string, unknown>): Logger
}

export function createLogger(opts: {
  level: LogLevel
  requestId: string
  environment: string
  bindings?: Record<string, unknown>
}): Logger {
  const threshold = RANK[opts.level]

  const emit = (level: LogLevel, msg: string, meta?: Record<string, unknown>) => {
    if (RANK[level] < threshold) return
    // Single-line JSON – parseable by any log aggregator.
    console.log(
      JSON.stringify({
        level,
        msg,
        ts: new Date().toISOString(),
        requestId: opts.requestId,
        env: opts.environment,
        ...opts.bindings,
        ...meta,
      })
    )
  }

  return {
    debug: (m, meta) => emit('debug', m, meta),
    info: (m, meta) => emit('info', m, meta),
    warn: (m, meta) => emit('warn', m, meta),
    error: (m, meta) => emit('error', m, meta),
    child: (bindings) =>
      createLogger({ ...opts, bindings: { ...opts.bindings, ...bindings } })
  }
}
```

****Never log the `Authorization` header, raw tokens, password fields, or PII.**** Maintain an explicit redaction list. Assume every log line will eventually be read by someone who should not see secrets.

9.4 Access Log

```
// core/middleware/logger.ts
import { createMiddleware } from 'hono/factory'
import type { AppEnv } from '@core/types'

export const accessLog = createMiddleware<AppEnv>(async (c, next) => {
  const start = Date.now()

  await next()

  const logger = c.get('logger')
  const durationMs = Date.now() - start
  const status = c.res.status

  const line = {
    method: c.req.method,
    path: c.req.path,
    status,
    durationMs,
    principalId: c.get('principal')?.id ?? null,
  }

  if (status >= 500) logger.error('http.request', line)
  else if (status >= 400) logger.warn('http.request', line)
  else logger.info('http.request', line)
})
```

9.5 CORS

```
// core/middleware/cors.ts
import { cors } from 'hono/cors'
import type { MiddlewareHandler } from 'hono'
import type { AppEnv } from '@core/types'

export function corsMiddleware(): MiddlewareHandler<AppEnv> {
  return async (c, next) => {
    // Origins come from env, which comes from the container.
    // But container middleware runs AFTER cors – so parse directly.
    const allowed = (c.env.CORS_ORIGINS as string)
      .split(',')
      .map((s) => s.trim())

    return cors({
      origin: (origin) => (allowed.includes(origin) ? origin : null),
      allowMethods: ['GET', 'POST', 'PATCH', 'PUT', 'DELETE', 'OPTIONS'],
      allowHeaders: [
        'Content-Type',
        'Authorization',
        'X-Request-Id',
        'Idempotency-Key',
      ],
      exposeHeaders: [
        'X-Request-Id',
        'X-RateLimit-Limit',
        'X-RateLimit-Remaining',
        'X-RateLimit-Reset',
        'Link',
      ],
      credentials: true,
      maxAge: 86400,
    })(c, next)
  }
}
```

****Never use `origin: '\*'` with `credentials: true`.**** Browsers reject the combination, and if they didn't, it would be a vulnerability. Echo back only origins present in your allowlist.

9.6 Security Headers

```
// core/middleware/security-headers.ts
import { secureHeaders } from 'hono/secure-headers'

export const securityHeaders = secureHeaders({
  strictTransportSecurity: 'max-age=63072000; includeSubDomains; preload',
  xContentTypeOptions: 'nosniff',
  xFrameOptions: 'DENY',
  referrerPolicy: 'no-referrer',
  crossOriginResourcePolicy: 'same-site',
  // A JSON API serves no scripts. Lock it down entirely.
  contentSecurityPolicy: { defaultSrc: ['"none"'], frameAncestors: ['"none"'] },
})
```

9.7 Rate Limiting

Fixed-window counter in Workers KV. Simple, adequate for most APIs. For strict guarantees under burst, use a Durable Object.

```
// core/middleware/rate-limit.ts
import { createMiddleware } from 'hono/factory'
import { RateLimitError } from '@core/errors'
import type { AppEnv } from '@core/types'

export interface RateLimitOptions {
  limit: number
  windowSeconds: number
  /** Defaults to principal id, falling back to client IP. */
  keyFn?: (c: Context<AppEnv>) => string
}

export function rateLimit(opts: RateLimitOptions) {
  return createMiddleware<AppEnv>(async (c, next) => {
    const { kv } = c.get('container')

    const identity =
      opts.keyFn?.(c) ??
      c.get('principal')?.id ??
      c.req.header('cf-connecting-ip') ??
      'anonymous'

    const window = Math.floor(Date.now() / 1000 / opts.windowSeconds)
    const key = `rl:${c.req.path}:${identity}:${window}`

    const current = Number((await kv.rateLimit.get(key)) ?? 0)
    const remaining = Math.max(0, opts.limit - current - 1)
    const resetAt = (window + 1) * opts.windowSeconds

    c.header('X-RateLimit-Limit', String(opts.limit))
    c.header('X-RateLimit-Remaining', String(remaining))
    c.header('X-RateLimit-Reset', String(resetAt))

    if (current >= opts.limit) {
      c.header('Retry-After', String(resetAt - Math.floor(Date.now() / 1000)))
      throw new RateLimitError(resetAt)
    }

    // Fire-and-forget the increment; do not block the response.
    c.executionCtx.waitUntil(
      kv.rateLimit.put(key, String(current + 1), {
        expirationTtl: opts.windowSeconds + 60,
      }),
    )

    await next()
  })
}
```

****This is not atomic.**** Two concurrent requests can both read `current = 4` and both write `5`. Under contention, users get slightly more than their quota. That is an acceptable trade for most APIs. If you need exactness — billing, abuse prevention — use a Durable Object with a single-threaded counter.

9.8 Idempotency

For unsafe methods, allow clients to safely retry. Required for payments; useful everywhere.

```

// core/middleware/idempotency.ts
import { createMiddleware } from 'hono/factory'
import { ConflictError } from '@core/errors'
import type { AppEnv } from '@core/types'

interface StoredResponse {
  status: number
  body: unknown
  fingerprint: string
}

export const idempotency = createMiddleware<AppEnv>(async (c, next) => {
  const key = c.req.header('Idempotency-Key')
  if (!key || ![ 'POST', 'PATCH', 'PUT' ].includes(c.req.method)) {
    return next()
  }

  const { kv } = c.get('container')
  const principalId = c.get('principal')?.id ?? 'anon'
  const storeKey = `idem:${principalId}:${key}`

  // Fingerprint the request so the same key with a different body is rejected.
  const rawBody = await c.req.raw.clone().text()
  const fingerprint = await sha256(`${c.req.method}:${c.req.path}:${rawBody}`)

  const cached = await kv.idempotency.get<StoredResponse>(storeKey, 'json')
  if (cached) {
    if (cached.fingerprint !== fingerprint) {
      throw new ConflictError(
        'Idempotency-Key was reused with a different request body',
      )
    }
    c.header('Idempotency-Replayed', 'true')
    return c.json(cached.body as object, cached.status as 200)
  }

  await next()

  // Only cache successful, deterministic outcomes.
  if (c.res.status < 300) {
    const body = await c.res.clone().json()
    c.executionCtx.waitUntil(
      kv.idempotency.put(
        storeKey,
        JSON.stringify({ status: c.res.status, body, fingerprint }),
        { expirationTtl: 60 * 60 * 24 },
      ),
    )
  }
})

async function sha256(input: string): Promise<string> {
  const buf = await crypto.subtle.digest('SHA-256', new TextEncoder().encode(input))
  return [...new Uint8Array(buf)].map((b) => b.toString(16).padStart(2, '0')).join('')
}

```

10. Authentication & Bearer Tokens

10.1 Two Token Types, One Principal

Real APIs authenticate two distinct kinds of caller:

	Session token	API key
Issued to	A human, via login	A machine, via dashboard
Lifetime	Short (hours/days), rotating	Long (months), manually revoked
Managed by	Better Auth	Your own <code>api_keys</code> table
Storage	Cookie or <code>Authorization: Bearer</code>	<code>Authorization: Bearer</code> only
Format	Opaque session ID	<code>sk_live_<id>_<secret></code>

Both must resolve to a single `Principal` type, so that every downstream layer — authorization, rate limiting, logging, business logic — is indifferent to how the caller authenticated.

10.2 The Principal

```
// auth/principal.ts

export type PrincipalKind = 'user' | 'service'

export interface Principal {
  /** Stable identity. For services, the owning user's id. */
  id: string
  kind: PrincipalKind
  email: string | null
  roles: readonly string[]
  scopes: readonly string[]
  /** Present only for `kind: 'session'`. Useful for revocation. */
  sessionId: string | null
  /** Present only for `kind: 'service'`. Useful for audit. */
  apiKeyId: string | null
}

export const isUser = (p: Principal): boolean => p.kind === 'user'
export const isService = (p: Principal): boolean => p.kind === 'service'
```

****This is the seam.**** Downstream code never asks "was this a cookie or an API key?" It asks "who is this, and what may they do?" Adding OAuth2, mTLS, or JWT later means writing one more resolver — nothing else changes.

10.3 Better Auth Configuration


```
// db/schema/api-keys.ts
export const apiKeys = pgTable(
  'api_keys',
  {
    id: uuid('id').primaryKey().defaultRandom(),
    lookupId: text('lookup_id').notNull(), // the 8-char segment
    secretHash: text('secret_hash').notNull(), // SHA-256(secret + pepper)
    userId: uuid('user_id').notNull().references(() => users.id, {
      onDelete: 'cascade',
    }),
    name: text('name').notNull(),
    scopes: text('scopes').array().notNull().default([]),
    lastUsedAt: timestamp('last_used_at', { withTimezone: true }),
    expiresAt: timestamp('expires_at', { withTimezone: true }),
    revokedAt: timestamp('revoked_at', { withTimezone: true }),
    createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  },
  (t) => ({
    lookupIdx: uniqueIndex('api_keys_lookup_idx').on(t.lookupId),
    userIdx: index('api_keys_user_idx').on(t.userId),
  }),
)

```

Service:

```
// auth/api-key.service.ts
import { ConflictError, UnauthorizedError } from '@core/errors'

const ALPHABET = 'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789'

function randomToken(len: number): string {
  const bytes = crypto.getRandomValues(new Uint8Array(len))
  return Array.from(bytes, (b) => ALPHABET[b % ALPHABET.length]).join('')
}

async function hashSecret(secret: string, pepper: string): Promise<string> {
  const buf = await crypto.subtle.digest(
    'SHA-256',
    new TextEncoder().encode(`${secret}${pepper}`),
  )
  return [...new Uint8Array(buf)].map((b) => b.toString(16).padStart(2, '0')).join('')
}

/** Constant-time comparison. Prevents timing oracles. */
function timingSafeEqual(a: string, b: string): boolean {
  if (a.length !== b.length) return false
  let diff = 0
  for (let i = 0; i < a.length; i++) diff |= a.charCodeAt(i) ^ b.charCodeAt(i)
  return diff === 0
}

export function makeApiKeyService(deps: {
  apiKeyRepo: ApiKeyRepository
  pepper: string
}) {
  const { apiKeyRepo, pepper } = deps

  return {
    /**
     * Returns the plaintext token EXACTLY ONCE.
     * It is never recoverable afterward.
     */
    async issue(input: {
      userId: string
      name: string
      scopes: string[]
      env: 'live' | 'test'
      expiresAt?: Date
    }): Promise<{ token: string; id: string }> {
      const lookupId = randomToken(8)
      const secret = randomToken(32)
      const token = `sk_${input.env}_${lookupId}_${secret}`

      const row = await apiKeyRepo.create({
        lookupId,
        secretHash: await hashSecret(secret, pepper),
        userId: input.userId,
        name: input.name,
        scopes: input.scopes,
        expiresAt: input.expiresAt ?? null,
      })

      return { token, id: row.id }
    },
    async verify(token: string): Promise<Principal | null> {
      const parts = token.split('_')
      if (parts.length !== 4 || parts[0] !== 'sk') return null

      const [, , lookupId, secret] = parts

      const key = await apiKeyRepo.findByLookupId(lookupId)
      if (!key) return null
      if (key.revokedAt) return null
      if (key.expiresAt && key.expiresAt < new Date()) return null

      const candidate = await hashSecret(secret, pepper)
      if (!timingSafeEqual(candidate, key.secretHash)) return null

      return {
        id: key.userId,
        kind: 'service',
        email: null,
        roles: [],
        scopes: key.scopes,
        sessionId: null,
        apiKeyId: key.id,
      }
    },
  }
}
}
```

****Why a pepper and not a salt?**** Salts defend against rainbow tables for *low-entropy* secrets (passwords). Our secret is 190 bits of CSPRNG output — rainbow tables are irrelevant. The pepper is an application-level secret stored outside the database, so a pure DB leak yields nothing. And because the secret is high-entropy, plain SHA-256 is correct — ****do not use bcrypt/argon2 here.**** They are slow by design, and you would pay that cost on every single API request for no security benefit.

10.6 The Auth Middleware

```
// core/middleware/auth.ts
import { createMiddleware } from 'hono/factory'
import { UnauthorizedError } from '@core/errors'
import type { AppEnv } from '@core/types'
import type { Principal } from '@auth/principal'

/**
 * Extracts `Bearer <token>` per RFC 6750 §2.1.
 * Case-insensitive scheme, exactly one space.
 */
function extractBearer(header: string | undefined): string | null {
  if (!header) return null
  const match = /^Bearer\s+(.+)$/i.exec(header.trim())
  return match?.[1] ?? null
}

async function resolvePrincipal(c: Context<AppEnv>): Promise<Principal | null> {
  const { auth, services } = c.get('container')
  const token = extractBearer(c.req.header('Authorization'))

  // --- Path 1: API key (machine caller) ---
  if (token?.startsWith('sk_')) {
    return services.apiKey.verify(token)
  }

  // --- Path 2: Session, via bearer OR cookie ---
  // Better Auth's bearer() plugin reads the header; getSession reads cookies.
  // Passing raw headers covers both in one call.
  const session = await auth.api.getSession({ headers: c.req.raw.headers })
  if (!session) return null

  return {
    id: session.user.id,
    kind: 'user',
    email: session.user.email,
    roles: (session.user as { roles?: string[] }).roles ?? ['user'],
    scopes: ['*'], // humans have full scope; roles gate them
    sessionId: session.session.id,
    apiKeyId: null,
  }
}

/** Hard gate. Rejects unauthenticated requests. */
export const requireAuth = createMiddleware<AppEnv>(async (c, next) => {
  const principal = await resolvePrincipal(c)

  if (!principal) {
    // RFC 6750 §3 – the WWW-Authenticate challenge is mandatory on 401.
    c.header('WWW-Authenticate', 'Bearer realm="api", error="invalid_token"')
    throw new UnauthorizedError('Missing or invalid credentials')
  }

  c.set('principal', principal)
  c.set('logger', c.get('logger').child({ principalId: principal.id }))
  await next()
})

/** Soft gate. Populates principal if present; never rejects. */
export const optionalAuth = createMiddleware<AppEnv>(async (c, next) => {
  c.set('principal', await resolvePrincipal(c).catch(() => null))
  await next()
})
})
```

10.7 The **WWW-Authenticate** Header

Most implementations omit this. It is required by RFC 6750, and well-behaved clients (and OpenAPI tooling) rely on it to distinguish failure modes:

Situation	Status	WWW-Authenticate
No credentials supplied	401	Bearer realm="api"
Malformed / unparseable token	401	Bearer realm="api", error="invalid_token"
Expired token	401	Bearer realm="api", error="invalid_token", error_description="expired"
Valid token, insufficient scope	403	Bearer realm="api", error="insufficient_scope", scope="posts:write"

****401** means "I don't know who you are." **403** means "I know who you are, and no." ****** Returning ``403`` for a missing token, or ``401`` for a scope failure, will break client retry logic.

11. Authorization

11.1 Scopes vs. Roles

- **Scopes** answer *what may this credential do?* They live on the token.
- **Roles** answer *what is this person?* They live on the user.

An admin user with a read-only API key must not be able to write. Therefore: **the effective permission is the intersection of role grants and token scopes.**

11.2 Scope Middleware

```
// core/middleware/require-scopes.ts
import { createMiddleware } from 'hono/factory'
import { ForbiddenError } from '@core/errors'
import type { AppEnv } from '@core/types'

export function requireScopes(...required: string[]) {
  return createMiddleware<AppEnv>(async (c, next) => {
    const principal = c.get('principal')
    if (!principal) throw new UnauthorizedError()

    const granted = new Set(principal.scopes)

    const satisfied = required.every(
      (scope) =>
        granted.has('*') ||
        granted.has(scope) ||
        // Wildcard: `posts:*` satisfies `posts:write`
        granted.has(`${scope.split(':')[0]}:*`),
    )

    if (!satisfied) {
      c.header(
        'WWW-Authenticate',
        `Bearer realm="api", error="insufficient_scope", scope="${required.join(' ')}"`,
      )
      throw new ForbiddenError('Insufficient scope', { required })
    }

    await next()
  })
}

export function requireRole(...roles: string[]) {
  return createMiddleware<AppEnv>(async (c, next) => {
    const principal = c.get('principal')
    if (!principal) throw new UnauthorizedError()
    if (!roles.some((r) => principal.roles.includes(r))) {
      throw new ForbiddenError('Insufficient role', { required: roles })
    }
    await next()
  })
}
```

11.3 Resource-Level Authorization Belongs in the Service

Middleware cannot answer "may this user edit *this specific post*?" — that requires fetching the post. Route-level authorization handles **coarse** access. **Fine-grained** ownership checks belong in the service, where the data lives.

```
// modules/post/post.service.ts
async update(postId: string, patch: UpdatePostInput, actor: Principal) {
  const post = await postRepo.findById(postId)
  if (!post) throw new NotFoundError('Post', postId)

  const isOwner = post.authorId === actor.id
  const isModerator = actor.roles.includes('moderator')

  if (!isOwner && !isModerator) {
    // 404, not 403 — do not confirm the resource exists to a
    // caller who has no right to know.
    throw new NotFoundError('Post', postId)
  }

  return postRepo.update(postId, patch)
}
```

****Return `404`, not `403`, for resources the caller may not access.**** A `403` confirms the resource exists. For anything enumerable — user IDs, private documents — that is an information leak. Reserve `403` for cases where existence is already public knowledge.

The service takes `actor: Principal` as an **explicit parameter**. It does not read a global "current user." This keeps P5 intact: the method is callable from a cron job by passing a system principal.

12. Error Handling

12.1 Domain Errors, Translated at the Edge

****A service must never throw `HTTPException`.**** It throws a `NotFoundError`. Whether that becomes a `404` is a decision for the HTTP layer — and a cron job consuming the same service has no use for status codes.

12.2 The Error Hierarchy

```
// core/errors.ts

export abstract class AppError extends Error {
  abstract readonly status: number
  /** Stable, machine-readable. Clients switch on this, not on `message`. */
  abstract readonly code: string
  /** Safe to serialize to the client. Never contains internals. */
  readonly details?: Record<string, unknown>

  constructor(message: string, details?: Record<string, unknown>) {
    super(message)
    this.name = new.target.name
    this.details = details
  }
}

export class ValidationError extends AppError {
  readonly status = 422
  readonly code = 'VALIDATION_FAILED'
}

export class UnauthorizedError extends AppError {
  readonly status = 401
  readonly code = 'UNAUTHORIZED'
  constructor(message = 'Authentication required') { super(message) }
}

export class ForbiddenError extends AppError {
  readonly status = 403
  readonly code = 'FORBIDDEN'
  constructor(message = 'Insufficient permissions', details?: Record<string, unknown>) {
    super(message, details)
  }
}

export class NotFoundError extends AppError {
  readonly status = 404
  readonly code = 'NOT_FOUND'
  constructor(resource: string, id?: string) {
    super(`${resource} not found`, id ? { resource, id } : { resource })
  }
}

export class ConflictError extends AppError {
  readonly status = 409
  readonly code = 'CONFLICT'
}

export class RateLimitError extends AppError {
  readonly status = 429
  readonly code = 'RATE_LIMITED'
  constructor(resetAt: number) {
    super('Rate limit exceeded', { resetAt })
  }
}

export class InternalError extends AppError {
  readonly status = 500
  readonly code = 'INTERNAL_ERROR'
  constructor(message = 'An unexpected error occurred') { super(message) }
}
```

12.3 The Error Envelope

Every error response, without exception, has this shape:

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Request body failed validation",
    "details": {
      "issues": [
        { "path": "email", "message": "Invalid email address", "code": "invalid_string" },
        { "path": "name", "message": "String must contain at least 1 character", "code": "too_small" }
      ]
    },
    "requestId": "9f2c1e4a-..."
  }
}
```

****`requestId` in every error response is the highest-leverage operational decision in this document.**** A user reports a bug. They paste the id. You grep your logs. You have the exact request, its principal, its duration, and its stack trace. This costs you one line of code and saves you hours per incident.

12.4 The Handler

```
// core/middleware/error-handler.ts
import { ZodError } from 'zod'
import { HTTPException } from 'hono/http-exception'
import { DrizzleError } from 'drizzle-orm'
import { AppError } from '@core/errors'
import type { AppEnv } from '@core/types'
import type { ErrorHandler, NotFoundHandler } from 'hono'

interface ErrorBody {
  error: {
    code: string
    message: string
    details?: Record<string, unknown>
    requestId: string
  }
}

const body = (
  code: string,
  message: string,
  requestId: string,
  details?: Record<string, unknown>,
): ErrorBody => ({ error: { code, message, requestId, ...(details && { details }) } })

export const errorHandler: ErrorHandler<AppEnv> = (err, c) => {
  const requestId = c.get('requestId') ?? 'unknown'
  const logger = c.get('logger')

  // 1. Our own domain errors – expected, safe to surface.
  if (err instanceof AppError) {
    logger?.warn('app.error', { code: err.code, status: err.status, msg: err.message })
    return c.json(body(err.code, err.message, requestId, err.details), err.status as 400)
  }

  // 2. Zod – validation failures from @hono/zod-openapi.
  if (err instanceof ZodError) {
    return c.json(
      body('VALIDATION_FAILED', 'Request failed validation', requestId, {
        issues: err.issues.map((i) => ({
          path: i.path.join('.'),
          message: i.message,
          code: i.code,
        })),
      }),
      422,
    )
  }

  // 3. Postgres constraint violations – translate to domain semantics.
  if (isPgError(err)) {
    switch (err.code) {
      case '23505': // unique violation
        return c.json(body('CONFLICT', 'Resource already exists', requestId), 409)
      case '23503': // foreign key violation
        return c.json(body('CONFLICT', 'Referenced resource does not exist', requestId), 409)
      case '23514': // check violation
        return c.json(body('VALIDATION_FAILED', 'Constraint violated', requestId), 422)
    }
  }

  // 4. Hono internals.
  if (err instanceof HTTPException) {
    return c.json(body('HTTP_ERROR', err.message, requestId), err.status)
  }

  // 5. Everything else. LOG THE FULL ERROR. LEAK NOTHING.
  logger?.error('unhandled.error', {
    message: err.message,
    stack: err.stack,
    name: err.name,
  })

  return c.json(body('INTERNAL_ERROR', 'An unexpected error occurred', requestId), 500)
}

export const notFoundHandler: NotFoundHandler<AppEnv> = (c) =>
  c.json(
    body('NOT_FOUND', `No route for ${c.req.method} ${c.req.path}`,
      c.get('requestId') ?? 'unknown'),
    404,
  )

function isPgError(e: unknown): e is { code: string } {
  return typeof e === 'object' && e !== null && 'code' in e &&
    typeof (e as { code: unknown }).code === 'string'
}
```

****Case 5 is the security boundary.**** An unhandled error may contain a connection string, a query with embedded PII, or a file path revealing your directory structure. Log it in full. Return a constant string. There is never a reason to send `err.message` from an unknown error to a client.

12.5 Postgres Error Codes Worth Handling

Code	Name	Maps to
23505	unique_violation	409 CONFLICT
23503	foreign_key_violation	409 CONFLICT
23502	not_null_violation	422 VALIDATION_FAILED
23514	check_violation	422 VALIDATION_FAILED
40001	serialization_failure	409 + advise retry
40P01	deadlock_detected	409 + advise retry
57014	query_canceled (timeout)	503

Handling 23505 in the error handler is your **safety net**, not your primary defense. Check for conflicts in the service (§8.2) so you can return a helpful, field-specific message. The DB constraint catches the race you didn't think about.

13. Pagination

13.1 Choose Deliberately

There are two viable strategies. They are not interchangeable, and shipping the wrong one is expensive to undo.

	Offset / Page	Cursor / Keyset
Query	<code>LIMIT n OFFSET (p-1)*n</code>	<code>WHERE (k) < (cursor) LIMIT n</code>
Complexity at page <i>p</i>	O(p·n) — Postgres reads and discards every skipped row	O(log N + n) — index seek
Jump to arbitrary page	✔ Yes	✗ No
Total count available	✔ Yes (extra query)	✗ Not without a scan
Stable under concurrent inserts	✗ No — rows shift between pages	✔ Yes
Correct for infinite scroll	✗ No	✔ Yes
Correct for a numbered admin table	✔ Yes	✗ No

The offset drift bug. User loads page 1 (rows 1–20). Someone inserts a new row at the top. User loads page 2 (`OFFSET 20``). What was row 20 is now row 21 — the user sees it **twice**, and what was row 40 is now row 41 and gets **skipped** when they reach page 3. This is not a hypothetical. It is the default behavior of every `OFFSET``-paginated feed under write load.

Default to cursor pagination for all list endpoints. Offer offset only where a numbered page control is a genuine product requirement — internal admin tables, mostly. Expose both from the same endpoint. Let the client's query parameters select the strategy.

13.2 Types

```
// core/pagination/types.ts

export interface PageMeta {
  page: number
  limit: number
  total: number
  totalPages: number
  hasNextPage: boolean
  hasPrevPage: boolean
}

export interface CursorMeta {
  limit: number
  nextCursor: string | null
  prevCursor: string | null
  hasNextPage: boolean
}

export interface Paginated<T> {
  data: T[]
  meta: PageMeta
}

export interface CursorPaginated<T> {
  data: T[]
  meta: CursorMeta
}

/** The decoded cursor. Must be a UNIQUE, ORDERED tuple. */
export interface Cursor {
  createdAt: Date
  id: string
}

export interface OffsetParams { page: number; limit: number }
export interface CursorParams { limit: number; cursor: Cursor | null }
```

13.3 Zod Schemas

```
// core/pagination/schema.ts
import { z } from '@hono/zod-openapi'

const limit = z.coerce
  .number()
  .int()
  .min(1)
  .max(100)
  .default(20)
  .openapi({ description: 'Items per page (1-100).', example: 20 })

export const OffsetQuerySchema = z.object({
  page: z.coerce.number().int().min(1).default(1)
    .openapi({ description: '1-indexed page number.', example: 1 }),
  limit,
  sort: z.enum(['createdAt', 'name']).default('createdAt').openapi({}),
  order: z.enum(['asc', 'desc']).default('desc').openapi({}),
})

export const CursorQuerySchema = z.object({
  cursor: z.string().optional().openapi({
    description: 'Opaque cursor from `meta.nextCursor`. Do not construct by hand.',
    example: 'eyJjIjoimjAyNS0wMS0xNVQxMDozMDowMFoiLCJpIjoimDES...',
  }),
  limit,
})

export type OffsetQuery = z.infer<typeof OffsetQuerySchema>
export type CursorQuery = z.infer<typeof CursorQuerySchema>

// --- Response envelopes, as OpenAPI-compatible generics ---

export const PageMetaSchema = z.object({
  page: z.number().int(),
  limit: z.number().int(),
  total: z.number().int(),
  totalPages: z.number().int(),
  hasNextPage: z.boolean(),
  hasPrevPage: z.boolean(),
}).openapi('PageMeta')

export const CursorMetaSchema = z.object({
  limit: z.number().int(),
  nextCursor: z.string().nullable(),
  prevCursor: z.string().nullable(),
  hasNextPage: z.boolean(),
}).openapi('CursorMeta')

/** Wraps any item schema into a paginated envelope. */
export function paginated<T extends z.ZodTypeAny>(item: T, name: string) {
  return z.object({ data: z.array(item), meta: PageMetaSchema }).openapi(name)
}

export function cursorPaginated<T extends z.ZodTypeAny>(item: T, name: string) {
  return z.object({ data: z.array(item), meta: CursorMetaSchema }).openapi(name)
}
```

13.4 The Cursor Itself

****A cursor must encode a strictly-ordered, unique tuple.**** ``createdAt`` alone is insufficient — two rows can share a timestamp, and you will silently skip or duplicate them. Always append a tiebreaker: ``(createdAt, id)``.

```
// core/pagination/helpers.ts
import { ValidationError } from '@core/errors'
import type { Cursor, CursorMeta } from './types'

/**
 * Cursors are base64url-encoded JSON. They are OPAQUE:
 * clients must treat them as strings and never parse them.
 * Encoding (not encryption) – do not put secrets in a cursor.
 */
export function encodeCursor(row: { createdAt: Date; id: string }): string {
  const json = JSON.stringify({ c: row.createdAt.toISOString(), i: row.id })
  return btoa(json).replace(/\+/g, '-').replace(/\//g, '_').replace(/=+$/, '')
}

export function decodeCursor(raw: string): Cursor {
  try {
    const b64 = raw.replace(/-/g, '+').replace(/_/g, '/')
    const parsed = JSON.parse(atob(b64)) as { c: string; i: string }

    const createdAt = new Date(parsed.c)
    if (Number.isNaN(createdAt.getTime())) throw new Error('bad date')
    if (typeof parsed.i !== 'string' || !parsed.i) throw new Error('bad id')

    return { createdAt, id: parsed.i }
  } catch {
    throw new ValidationError('Malformed cursor', { field: 'cursor' })
  }
}

export function toCursorMeta<T extends { createdAt: Date; id: string }>(
  rows: T[],
  hasNextPage: boolean,
): CursorMeta {
  return {
    limit: rows.length,
    nextCursor: hasNextPage && rows.length > 0
      ? encodeCursor(rows[rows.length - 1])
      : null,
    prevCursor: rows.length > 0 ? encodeCursor(rows[0]) : null,
    hasNextPage,
  }
}
```

13.5 The Keyset Predicate

For `ORDER BY created_at DESC, id DESC`, the "rows after this cursor" predicate is:

```
WHERE (created_at, id) < ($1, $2)
ORDER BY created_at DESC, id DESC
LIMIT $3
```

Postgres supports native **row-value comparison**, and it is index-optimizable. Drizzle does not yet expose it ergonomically, so express it either as raw SQL or as its logical expansion:

```
// Expansion (portable, what 57.2 uses):
or(
  lt(users.createdAt, cursor.createdAt),
  and(eq(users.createdAt, cursor.createdAt), lt(users.id, cursor.id)),
)

// Row-value (faster; single index scan):
sql`${users.createdAt}, ${users.id} < (${cursor.createdAt}, ${cursor.id})`
```

****Required index:****

```
CREATE INDEX users_cursor_idx ON users (created_at DESC, id DESC);
```

The index column order and direction must match the ``ORDER BY`` exactly. Without it, Postgres sorts the whole table every page.

13.6 The `limit + 1` Trick

To know whether a next page exists, fetch one more row than you need:

```
const rows = await repo.paginateCursor({ limit: q.limit + 1, cursor })
const hasMore = rows.length > q.limit
const page = hasMore ? rows.slice(0, q.limit) : rows
```

This costs one extra row. The alternative — `SELECT count(*)` — costs a full table scan on every request. Never do that for cursor pagination.

13.7 Response Shape

Cursor:

```
{
  "data": [ { "id": "01J...", "name": "Ada" } ],
  "meta": {
    "limit": 20,
    "nextCursor": "eyJjIjoimjAyNS0wMS0xNVQxMDozMDowMFoiLCJpIjoimDFKLi4uIn0",
    "prevCursor": "eyJjIjoimjAyNS0wMS0xNVQwODoxMjowMFoiLCJpIjoimDFLLi4uIn0",
    "hasNextPage": true
  }
}
```

Offset:

```
{
  "data": [ { "id": "01J...", "name": "Ada" } ],
  "meta": {
    "page": 2, "limit": 20, "total": 137,
    "totalPages": 7, "hasNextPage": true, "hasPrevPage": true
  }
}
```

13.8 Link Headers (RFC 8288)

Machine-readable navigation. Cheap to add; makes your API scriptable.

```
// core/pagination/link-header.ts
export function buildLinkHeader(
  url: URL,
  meta: { nextCursor: string | null; prevCursor: string | null },
): string | null {
  const links: string[] = []
  const withParam = (k: string, v: string) => {
    const u = new URL(url)
    u.searchParams.set(k, v)
    return u.toString()
  }

  if (meta.nextCursor) links.push(`<${withParam('cursor', meta.nextCursor)}>; rel="next"`)
  if (meta.prevCursor) links.push(`<${withParam('cursor', meta.prevCursor)}>; rel="prev"`)

  return links.length ? links.join(', ') : null
}
```

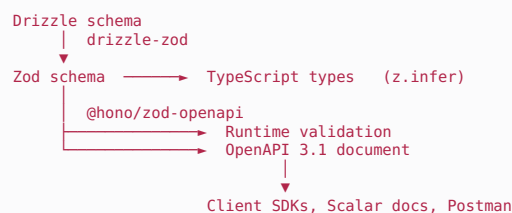
```
Link: <https://api.example.com/v1/users?cursor=eyJj...&limit=20>; rel="next"
X-Total-Count: 137
```

13.9 Guardrails

- **Clamp limit server-side.** `.max(100)` in Zod. A client asking for `limit=1000000` gets a `422`, not an OOM.
- **Never trust a cursor.** It is user input. Validate on decode, as §13.4 does.
- **Never encrypt a cursor to hide fields.** If a field must not be exposed, do not put it in the cursor.
- **Sort keys must be indexed.** An unindexed `sort=name` is a table scan wearing a query parameter's clothes. Whitelist sortable columns with `z.enum()` and ensure each has an index.

14. OpenAPI & Route Definitions

14.1 The Chain of Truth



****This is the one place we choose implicit over explicit — deliberately.**** The DB schema is the source of truth. Deriving validators from it means drift is ***structurally impossible*** rather than merely discouraged. Writing them by hand means the day someone adds a column, the validator silently omits it.

14.2 Deriving Schemas

```
// modules/user/user.schema.ts
import { createSelectSchema, createInsertSchema } from 'drizzle-zod'
import { z } from '@hono/zod-openapi'
import { users, type UserRow } from '@db/schema'

const base = createSelectSchema(users)

/** What we send to clients. Note the omissions. */
export const UserSchema = base
  .omit({ deletedAt: true })
  .extend({
    createdAt: z.coerce.date().transform((d) => d.toISOString()),
    updatedAt: z.coerce.date().transform((d) => d.toISOString()),
  })
  .openapi('User', {
    example: {
      id: '01JQZX3F8K2M9N4P5R6S7T8V9W',
      email: 'ada@example.com',
      name: 'Ada Lovelace',
      emailVerified: true,
      avatarUrl: null,
      createdAt: '2025-01-15T10:30:00.000Z',
      updatedAt: '2025-01-15T10:30:00.000Z',
    },
  })

export const CreateUserSchema = createInsertSchema(users, {
  email: (s) => s.email('Must be a valid email address'),
  name: (s) => s.min(1).max(120),
})
  .pick({ email: true, name: true })
  .openapi('CreateUser')

export const UpdateUserSchema = CreateUserSchema.partial()
  .refine((o) => Object.keys(o).length > 0, {
    message: 'At least one field must be provided',
  })
  .openapi('UpdateUser')

export const IdParamSchema = z.object({
  id: z.string().uuid().openapi({
    param: { name: 'id', in: 'path' },
    example: '01JQZX3F8K2M9N4P5R6S7T8V9W',
  }),
})

export type User = z.infer<typeof UserSchema>
export type CreateUserInput = z.infer<typeof CreateUserSchema>
export type UpdateUserInput = z.infer<typeof UpdateUserSchema>

/** Row → wire. The one place secrets get stripped. */
export const toPublicUser = (row: UserRow): User => UserSchema.parse(row)
```

****`toPublicUser` is a security control, not a formality.**** Route it through `.parse()` on a schema that omits sensitive columns. Then adding a ``passwordHash`` column to the table cannot accidentally leak it — Zod strips unknown keys by default. Never return a raw ``UserRow`` from a handler.

14.3 App Factory & Security Schemes

```
// core/http/openapi.ts
import { OpenAPIHono } from '@hono/zod-openapi'
import { ZodError } from 'zod'
import type { AppEnv } from '@core/types'

export function createApp() {
  return new OpenAPIHono<AppEnv>({
    // Route validation failures through OUR error envelope.
    defaultHook: (result, c) => {
      if (!result.success) throw result.error as ZodError
    },
  })
}

export function registerOpenApi(app: OpenAPIHono<AppEnv>) {
  app.openapiRegistry.registerComponent('securitySchemes', 'bearerAuth', {
    type: 'http',
    scheme: 'bearer',
    bearerFormat: 'opaque',
    description:
      'Session token from `POST /api/auth/sign-in`, or an API key (`sk_live_...`).'
  })

  app.doc31('/openapi.json', {
    openapi: '3.1.0',
    info: {
      title: 'API',
      version: '1.0.0',
      description: 'Reference implementation.',
    },
    servers: [{ url: 'https://api.example.com', description: 'Production' }],
    security: [{ bearerAuth: [] }], // default; overridden per-route
  })
}
```

14.4 Reusable Error Responses

Do not repeat `401` / `404` / `422` definitions across forty routes.

```
// core/http/responses.ts
import { z } from '@hono/zod-openapi'

export const ErrorSchema = z.object({
  error: z.object({
    code: z.string(),
    message: z.string(),
    details: z.record(z.unknown()).optional(),
    requestId: z.string(),
  }),
}).openapi('Error')

const json = (description: string) => ({
  content: { 'application/json': { schema: ErrorSchema } },
  description,
})

export const errorResponses = {
  400: json('Bad request'),
  401: json('Missing or invalid credentials'),
  403: json('Insufficient permissions'),
  404: json('Resource not found'),
  409: json('Conflict with current state'),
  422: json('Request failed validation'),
  429: json('Rate limit exceeded'),
  500: json('Internal server error'),
} as const

/** Pick only the errors a given route can actually produce. */
export const errs = <K extends keyof typeof errorResponses>(...codes: K[]) =>
  Object.fromEntries(codes.map((c) => [c, errorResponses[c]])) as Pick<
    typeof errorResponses, K
  >
```

14.5 Routes

```
// modules/user/user.routes.ts
import { createRoute, z } from '@hono/zod-openapi'
import { createApp } from '@core/http/openapi'
import { errs } from '@core/http/responses'
import { requireAuth } from '@core/middleware/auth'
import { requireScopes } from '@core/middleware/require-scopes'
import { rateLimit } from '@core/middleware/rate-limit'
import {
  CursorQuerySchema, OffsetQuerySchema, cursorPaginated, paginated,
} from '@core/pagination/schema'
import { buildLinkHeader } from '@core/pagination/link-header'
import {
  UserSchema, CreateUserSchema, UpdateUserSchema, IdParamSchema,
} from './user.schema'

export const userRoutes = createApp()

// — GET /users —————
const listRoute = createRoute({
  method: 'get',
  path: '/users',
  tags: ['Users'],
  summary: 'List users',
  description:
    'Supports both cursor (default, stable) and offset (`?page=`) pagination. ' +
```

```

    'Prefer cursor for feeds and infinite scroll.',
    security: [{ bearerAuth: [] }],
    middleware: [
      requireAuth,
      requireScopes('users:read'),
      rateLimit({ limit: 100, windowSeconds: 60 }),
    ] as const,
    request: {
      query: z.union([
        CursorQuerySchema,
        OffsetQuerySchema.extend({ page: z.coerce.number().int().min(1) }),
      ]),
    },
    responses: {
      200: {
        content: {
          'application/json': {
            schema: z.union([
              cursorPaginated(UserSchema, 'UserCursorPage'),
              paginated(UserSchema, 'UserOffsetPage'),
            ]),
          },
        },
        description: 'A page of users.',
        headers: z.object({
          Link: z.string().optional().openapi({ description: 'RFC 8288 navigation.' }),
        }),
      },
      ...errs(401, 403, 422, 429),
    },
  },
})

userRoutes.openapi(listRoute, async (c) => {
  const { services } = c.get('container')
  const q = c.req.valid('query')

  // Presence of `page` selects the strategy.
  if ('page' in q) {
    return c.json(await services.user.listOffset(q), 200)
  }

  const result = await services.user.listCursor(q)
  const link = buildLinkHeader(new URL(c.req.url), result.meta)
  if (link) c.header('Link', link)

  return c.json(result, 200)
})

// — GET /users/:id —————
const getRoute = createRoute({
  method: 'get',
  path: '/users/{id}',
  tags: ['Users'],
  summary: 'Get a user by ID',
  security: [{ bearerAuth: [] }],
  middleware: [requireAuth, requireScopes('users:read')] as const,
  request: { params: IdParamSchema },
  responses: {
    200: {
      content: { 'application/json': { schema: UserSchema } },
      description: 'The user.',
    },
    ...errs(401, 403, 404),
  },
})

userRoutes.openapi(getRoute, async (c) => {
  const { services } = c.get('container')
  const { id } = c.req.valid('param')
  return c.json(await services.user.getById(id), 200)
})

// — POST /users —————
const createUserRoute = createRoute({
  method: 'post',
  path: '/users',
  tags: ['Users'],
  summary: 'Create a user',
  security: [{ bearerAuth: [] }],
  middleware: [requireAuth, requireScopes('users:write')] as const,
  request: {
    headers: z.object({
      'idempotency-key': z.string().uuid().optional().openapi({
        description: 'Safely retry this request without creating duplicates.',
      }),
    }),
    body: {
      content: { 'application/json': { schema: CreateUserSchema } },
      required: true,
    },
  },
  responses: {
    201: {
      content: { 'application/json': { schema: UserSchema } },
      description: 'Created.',
      headers: z.object({ Location: z.string() }),
    },
    ...errs(401, 403, 409, 422),
  },
})

userRoutes.openapi(createUserRoute, async (c) => {
  const { services } = c.get('container')
  const input = c.req.valid('json')

  const user = await services.user.create(input)

```

```

    c.header('Location', `/v1/users/${user.id}`)
    return c.json(user, 201)
  })

// — PATCH /users/:id —
const updateRoute = createRoute({
  method: 'patch',
  path: '/users/{id}',
  tags: ['Users'],
  summary: 'Update a user',
  security: [{ bearerAuth: [] }],
  middleware: [requireAuth, requireScopes('users:write')] as const,
  request: {
    params: IdParamSchema,
    body: { content: { 'application/json': { schema: UpdateUserSchema } }, required: true },
  },
  responses: {
    200: { content: { 'application/json': { schema: UserSchema } }, description: 'Updated.' },
    ...errs(401, 403, 404, 422),
  },
})

userRoutes.openapi(updateRoute, async (c) => {
  const { services } = c.get('container')
  const { id } = c.req.valid('param')
  return c.json(await services.user.update(id, c.req.valid('json')), 200)
})

// — DELETE /users/:id —
const deleteRoute = createRoute({
  method: 'delete',
  path: '/users/{id}',
  tags: ['Users'],
  summary: 'Delete a user',
  security: [{ bearerAuth: [] }],
  middleware: [requireAuth, requireScopes('users:write')] as const,
  request: { params: IdParamSchema },
  responses: {
    204: { description: 'Deleted. No content.' },
    ...errs(401, 403, 404),
  },
})

userRoutes.openapi(deleteRoute, async (c) => {
  const { services } = c.get('container')
  await services.user.remove(c.req.valid('param').id)
  return c.body(null, 204)
})

```

****Observe the handlers.**** Every one is two to four lines: get the container, get validated input, call one service method, serialize. There is not a single `if` statement about business rules. This is P2 realized.

14.6 The Entrypoint

```

// src/index.ts
import { swaggerUI } from '@hono/swagger-ui'
import { createApp, registerOpenApi } from '@core/http/openapi'
import { requestId } from '@core/middleware/request-id'
import { accessLog } from '@core/middleware/logger'
import { securityHeaders } from '@core/middleware/security-headers'
import { corsMiddleware } from '@core/middleware/cors'
import { containerMiddleware } from '@core/middleware/container'
import { errorHandler, notFoundHandler } from '@core/middleware/error-handler'
import { authRouter } from '@auth/routes'
import { userRoutes } from '@modules/user'
import { postRoutes } from '@modules/post'

const app = createApp()

// — Global pipeline. ORDER IS LOAD-BEARING (see §9.1). —
app.use('*', requestId)
app.onError(errorHandler)
app.notFound(notFoundHandler)
app.use('*', securityHeaders)
app.use('*', corsMiddleware())
app.use('*', containerMiddleware)
app.use('*', accessLog)

// — Unauthenticated —
app.get('/health', (c) => c.json({ ok: true, ts: Date.now() }))
app.route('/api/auth', authRouter)

// — Versioned API. Auth is applied per-route, not globally,
//   so individual routes can opt out cleanly. —
app.route('/v1', userRoutes)
app.route('/v1', postRoutes)

// — Documentation —
registerOpenApi(app)
app.get('/docs', swaggerUI({ url: '/openapi.json' }))

export default app

// Scheduled work reuses the SAME services. No HTTP involved. (P5)
export const scheduled: ExportedHandler['scheduled'] = async (_e, env, ctx) => {
  const { createContainer } = await import('@container')
  const container = createContainer(env as never, { requestId: crypto.randomUUID() })
  ctx.waitUntil(container.services.user.purgeExpiredSoftDeletes())
}

```

Part IV — Operations

15. Testing Strategy

15.1 The Payoff of Explicit Dependencies

Because nothing imports a live connection, testing requires no mocking framework, no test container, and no module interception.

```
// modules/user/user.service.test.ts
import { describe, it, expect, vi } from 'vitest'
import { makeUserService } from './user.service'
import { NotFoundError, ConflictError } from '@core/errors'
import type { UserRepository } from './user.repository'

const noopLogger = {
  debug: vi.fn(), info: vi.fn(), warn: vi.fn(), error: vi.fn(),
  child: () => noopLogger,
}

const fixture = {
  id: '01JQZX', email: 'ada@example.com', name: 'Ada',
  emailVerified: true, avatarUrl: null,
  createdAt: new Date('2025-01-15'), updatedAt: new Date('2025-01-15'),
  deletedAt: null,
}

/** A partial repo is enough — structural typing does the rest. */
const repo = (over: Partial<UserRepository>): UserRepository => {
  ({ findById: async () => null, findByEmail: async () => null,
    create: async () => fixture, ...over }) as UserRepository

  describe('UserService', () => {
    it('throws NotFoundError when the user does not exist', async () => {
      const svc = makeUserService({
        userRepo: repo({ findById: async () => null }),
        logger: noopLogger,
      })
      await expect(svc.getById('missing')).rejects.toThrow(NotFoundError)
    })

    it('rejects duplicate emails before hitting the DB constraint', async () => {
      const create = vi.fn()
      const svc = makeUserService({
        userRepo: repo({ findByEmail: async () => fixture, create }),
        logger: noopLogger,
      })

      await expect(
        svc.create({ email: 'ada@example.com', name: 'Ada' }),
      ).rejects.toThrow(ConflictError)

      expect(create).not.toHaveBeenCalled()
    })

    it('normalizes email casing and trims whitespace', async () => {
      const create = vi.fn().mockResolvedValue(fixture)
      const svc = makeUserService({ userRepo: repo({ create }), logger: noopLogger })

      await svc.create({ email: ' ADA@Example.COM ', name: ' Ada ' })

      expect(create).toHaveBeenCalledWith({ email: 'ada@example.com', name: 'Ada' })
    })
  })
})
```

****No `vi.mock()`. No `Test.createTestingModule()`. No DI container.**** Twelve lines of setup, and every business rule is under test. If a service is ever hard to test, the diagnosis is always the same: a dependency was not injected.

15.2 The Pyramid

Level	What it covers	Doubles	Speed
Unit	Service business rules	Fake repos (plain objects)	~1ms
Integration	Repository ↔ real Postgres	Neon branch / Testcontainers	~50ms
Contract	Route ↔ OpenAPI schema	<code>app.request()</code> , no network	~5ms
E2E	Deployed worker	Nothing	~500ms

Weight heavily toward unit tests of the **service layer** — that is where your logic lives, and it is where bugs cost the most.

15.3 Route Contract Tests

Hono apps are callable without a server. Use it.

```
// modules/user/user.routes.test.ts
import { describe, it, expect } from 'vitest'
import app from '@/index'

describe('GET /v1/users', () => {
  it('returns 401 with a WWW-Authenticate challenge when unauthenticated', async () => {
    const res = await app.request('/v1/users', {}, testEnv)

    expect(res.status).toBe(401)
    expect(res.headers.get('WWW-Authenticate')).toMatch(/^Bearer realm="api"/)

    const body = await res.json()
    expect(body.error.code).toBe('UNAUTHORIZED')
    expect(body.error.requestId).toBeTruthy()
  })

  it('rejects limit above the server maximum', async () => {
    const res = await app.request('/v1/users?limit=99999',
      { headers: { Authorization: `Bearer ${testToken}` } }, testEnv)

    expect(res.status).toBe(422)
    const body = await res.json()
    expect(body.error.code).toBe('VALIDATION_FAILED')
    expect(body.error.details.issues[0].path).toBe('limit')
  })

  it('returns a stable cursor that round-trips', async () => {
    const first = await app.request('/v1/users?limit=2',
      { headers: { Authorization: `Bearer ${testToken}` } }, testEnv)
    const { meta } = await first.json()

    const second = await app.request(`/v1/users?limit=2&cursor=${meta.nextCursor}`,
      { headers: { Authorization: `Bearer ${testToken}` } }, testEnv)

    expect(second.status).toBe(200)
    const page2 = await second.json()
    // No overlap between pages.
    expect(page2.data[0].id).not.toBe(meta.data?.[0]?.id)
  })
})
```

15.4 Architecture Tests

Enforce the layer contract (§3.3) mechanically. This is the difference between a rule and a *guarantee*.

```
// .eslintrc - import boundaries
{
  "rules": {
    "import/no-restricted-paths": ["error", {
      "zones": [
        {
          "target": "./src/core",
          "from": "./src/modules",
          "message": "core/ must not depend on modules/ (R3)"
        },
        {
          "target": "./src/modules/**/*.service.ts",
          "from": "./src/db",
          "except": ["/src/db/schema"],
          "message": "Services must not touch the database directly (R2)"
        },
        {
          "target": "./src/modules/**/*.routes.ts",
          "from": "./src/modules/**/*.repository.ts",
          "message": "Routes must go through services, never repositories"
        }
      ]
    }
  ]
}
```

Plus the grep test from §3.3, in CI:

```
#!/usr/bin/env bash
# scripts/check-layers.sh
set -euo pipefail

if grep -rLE "\b(Context|c\.req|c\.json|HTTPException)\b" \
  src/modules/**/*.service.ts 2>/dev/null; then
  echo "FAIL: HTTP concerns leaked into the service layer (P5)"
  exit 1
fi

echo "OK: service layer is transport-agnostic"
```

16. Deployment

16.1 Migration Discipline

****Migrations are generated, reviewed, committed, and applied by CI — never by `db:push` against production.**** `drizzle-kit push` is a development convenience. It diffs and applies without an audit trail. It has no down migration. Using it in production is how you drop a column at 3am.

```
// package.json
{
  "scripts": {
    "db:generate": "drizzle-kit generate", // schema → SQL file. Commit it.
    "db:migrate": "drizzle-kit migrate", // apply. CI only.
    "db:studio": "drizzle-kit studio",
    "db:push": "drizzle-kit push" // LOCAL DEV ONLY
  }
}
```

16.2 Expand / Contract

Zero-downtime schema changes require two deploys, always:

```
Deploy 1 – EXPAND
  • Add the new nullable column.
  • Write to BOTH old and new columns.
  • Read from the OLD column.
  — Now the old and new code both work. Safe to roll back. —

Backfill
  • Populate the new column for existing rows, in batches.

Deploy 2 – MIGRATE READS
  • Read from the NEW column.
  — Verify. Old column is now unused but present. —

Deploy 3 – CONTRACT
  • Stop writing the old column.
  • Drop it.
```

Never combine expand and contract in one deploy. During a rolling deploy, old and new worker versions serve traffic simultaneously; a dropped column means the old version starts throwing.

16.3 CI/CD

```

# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: oven-sh/setup-bun@v2
      - run: bun install --frozen-lockfile

      - name: Typecheck
        run: bun tsc --noEmit

      - name: Lint (includes architecture boundaries)
        run: bun eslint src

      - name: Layer contract check
        run: bash scripts/check-layers.sh

      - name: Unit + contract tests
        run: bun test

      - name: Verify migrations are committed
        run: |
          bun db:generate
          git diff --exit-code drizzle/ \
          || (echo "Uncommitted migration! Run bun db:generate." && exit 1)

  deploy:
    needs: verify
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: oven-sh/setup-bun@v2
      - run: bun install --frozen-lockfile

      # Migrations BEFORE the worker. Expand/contract makes this safe.
      - name: Apply migrations
        run: bun db:migrate
        env:
          DATABASE_URL: ${ secrets.DATABASE_URL }

      - name: Deploy worker
        uses: cloudflare/wrangler-action@v3
        with:
          apiToken: ${ secrets.CF_API_TOKEN }
          command: deploy --env production

      - name: Smoke test
        run: |
          curl -sf https://api.example.com/health | jq -e '.ok == true'

```

16.4 Preview Environments

Neon branching makes per-PR databases nearly free — they are copy-on-write.

```

preview:
  if: github.event_name == 'pull_request'
  steps:
    - name: Create Neon branch
      id: branch
      uses: neondatabase/create-branch-action@v5
      with:
        project_id: ${ secrets.NEON_PROJECT_ID }
        branch_name: pr-${ github.event.number }
        api_key: ${ secrets.NEON_API_KEY }

    - run: bun db:migrate
      env:
        DATABASE_URL: ${ steps.branch.outputs.db_url }

    - run: bunx wrangler versions upload

```

17. Observability

17.1 The Three Questions

Every incident reduces to one of these. Your instrumentation exists to answer them in under a minute.

1. **Is it broken?** → error rate, `5xx` count, health check
2. **What is broken?** → structured logs filtered by `code` and `path`
3. **Why is it broken?** → the `requestId` from the user's screenshot

17.2 Correlation

The `requestId` established in §9.2 must appear in:

- ✓ every log line (via the child logger)
- ✓ every error response body (§12.3)
- ✓ the `X-Request-Id` response header
- ✓ any outbound HTTP call your service makes

That is the entire tracing story for a system this size. Do not reach for OpenTelemetry until you have more than one service.

17.3 What to Instrument

```
// The minimum viable set.
logger.info('http.request', { method, path, status, durationMs, principalId })
logger.warn('app.error',    { code, status, path })
logger.error('unhandled.error', { message, stack, name })
logger.info('db.slow query',   { durationMs, table }) // > 100ms
logger.info('auth.failed',    { reason, ip })        // never the token
```

17.4 Redaction

Maintain an explicit denylist. Assume every log line is eventually read by someone who should not see secrets.

```
const REDACTED = new Set([
  'password', 'passwordHash', 'token', 'secret', 'secretHash',
  'authorization', 'cookie', 'apiKey', 'sessionToken',
])

function redact(obj: Record<string, unknown>): Record<string, unknown> {
  return Object.fromEntries(
    Object.entries(obj).map(([k, v]) =>
      REDACTED.has(k.toLowerCase()) ? [k, '[REDACTED]'] : [k, v],
    ),
  ),
}
}
```

Appendices

Appendix A — Package Manifest

```
{
  "dependencies": {
    "hono": "^4",
    "@hono/zod-openapi": "^0.18",
    "@hono/swagger-ui": "^0.5",
    "zod": "^3",

    "drizzle-orm": "^0.38",
    "drizzle-zod": "^0.6",
    "@neondatabase/serverless": "^0.10",

    "better-auth": "^1"
  },
  "devDependencies": {
    "drizzle-kit": "^0.30",
    "wrangler": "^3",
    "@cloudflare/workers-types": "^4",
    "vitest": "^2",
    "@cloudflare/vitest-pool-workers": "^0.5",
    "typescript": "^5",
    "eslint": "^9",
    "eslint-plugin-import": "^2"
  }
}
```

tsconfig.json essentials:

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true, // catches rows[0] being undefined
    "exactOptionalPropertyTypes": true,
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "types": ["@cloudflare/workers-types"],
    "paths": { "@/*": [".src/*"] }
  }
}
```

noUncheckedIndexedAccess is the single highest-value flag in this list. It forces you to handle const [row] = await db.select()... returning undefined — which is exactly the case that produces Cannot read property 'id' of undefined in production.

Appendix B — Framework Comparison Matrix

Concern	NestJS	Laravel	Django	Go (chi)	This
Dependency wiring	DI container + decorators	Service container + facades	Implicit imports	Manual, explicit	Manual, one file
Dep graph visible?	✗ Distributed	✗ Magic	✗ Implicit	✓	✓ One file
Code organization	Modules	Layers	Apps	Packages	Modules
Validation	class-validator	Form Requests	Forms/DRF serializers	Manual	Zod
Schema → types	Manual DTOs	Manual	Auto (ORM)	Manual	Auto (drizzle-zod)
OpenAPI	Decorators	Manual/pkg	drf-spectacular	Manual/pkg	Derived from Zod
Test setup	Testing module	App boot	Test client	Plain structs	Plain objects
Startup cost	High (reflection)	High	High	~0	~0
Cold-start viable	✗	✗	✗	✓	✓
Refactor safety	Runtime tokens	Runtime strings	Runtime	Compile-time	Compile-time

Appendix C — Decision Log

Record decisions with their rationale, so future-you can tell a deliberate choice from an accident.

#	Decision	Rationale	Revisit when
D1	No DI container; explicit composition root	Shallow graph; isolate lifecycle; legibility	Graph exceeds ~30 nodes, or runtime-swappable impls needed
D2	Factory functions over classes	Structural typing; no <code>this</code> hazards; tree-shakeable	Team standardizes on OO
D3	Derive repo types via <code>ReturnType</code>	No drift; no double maintenance	A second implementation appears
D4	Cursor pagination by default	Offset drifts under concurrent writes; O(p·n)	Never — offer offset <i>alongside</i> , not instead
D5	<code>neon-http</code> driver	Stateless; safe to build per-request	Interactive transactions required → switch to WebSocket + pooling (§8.3)
D6	SHA-256 + pepper for API keys	Secret is 190-bit CSPRNG; bcrypt cost is pure waste per-request	Never for high-entropy secrets
D7	Throw domain errors; translate at edge	Services must run without HTTP (P5)	Consider <code>Result<T, E></code> — but decide <i>now</i> , migration is painful
D8	Modules as vertical slices	Deletable units; one folder per feature	Never
D9	<code>404</code> not <code>403</code> for unauthorized resources	Prevents existence enumeration	Existence is already public
D10	Zod validators derived from Drizzle schema	Structurally prevents drift (P4)	Never

Appendix D — The Two Decisions to Make Before Writing Code

D-1. Interfaces at boundaries: derive or declare?

- **Derive** (`ReturnType<typeof make...>`) — start here. Faster, no drift, still fully testable via structural typing.
- **Declare** (`interface UserRepository`) — extract the first time you need a second implementation.

Premature interface extraction costs you double maintenance for swappability you may never use. The Go instinct is right: *interfaces are defined by the consumer, when a second implementation appears*.

D-2. Errors: `throw` or `Result<T, E>` ?

- `throw` — idiomatic TypeScript, composes cleanly with `app.onError`, minimal ceremony. What this document uses.
- `Result<T, E>` (neverthrow, effect) — exhaustive error handling enforced by the type system. No invisible control flow.

****Consistency matters more than the choice.**** Pick one now. Migrating a codebase between them later means touching every service method, every handler, and every test. This is the one decision in this document that is genuinely expensive to reverse.

Summary ****Modules**** give you NestJS's structure. ****Factory functions**** give you its testability. ****Explicit wiring**** gives you something NestJS never had — the ability to open one file and see how the whole thing fits together. The layers hold because the direction of dependency never reverses: routes know about services, services know about repositories, repositories know about the database, and nothing knows about anything above it. Enforce it in the linter. Grep for it in CI. It will hold for years.